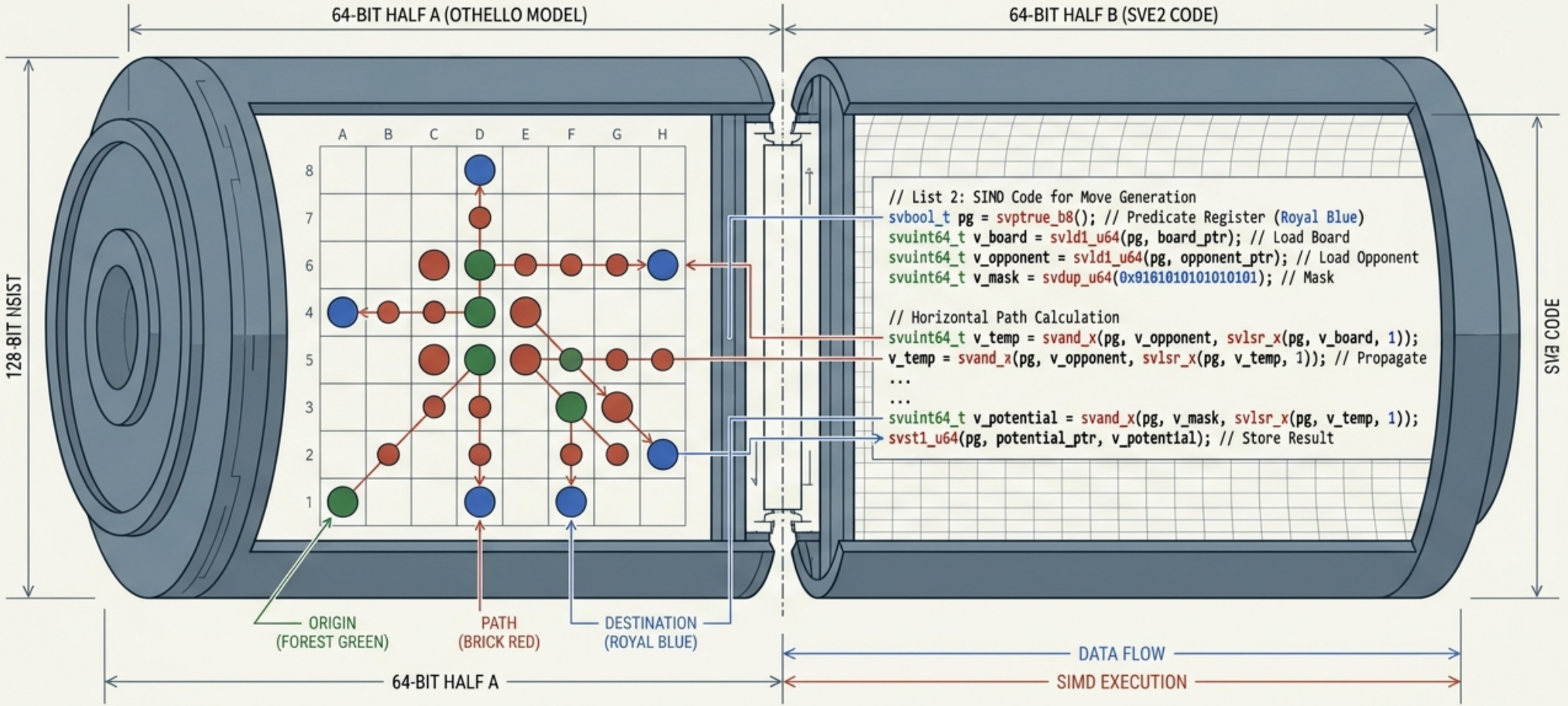
SVE2 オセロ合法手生成の完全解剖

抽象的なSIMDコード(List 2)を視覚的な幾何学モデルへ翻訳する



NotebookLM

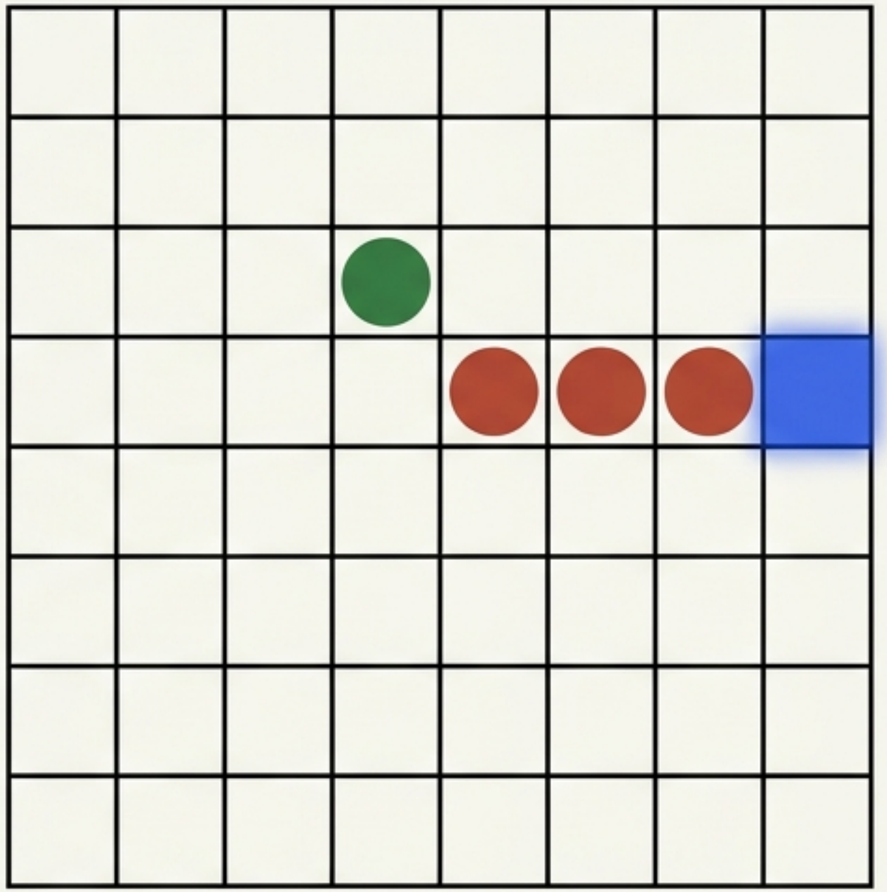
魔法ではなく、ただの幾何学

```
// SVE2 C++ Intrinsics Implementation
// Othello move generation logic

void sse2_move_gen(...) {
  svint8_t v_my, v_opp, v_mask;
  svbool_t pg1, pg2;
  // ... initialization ...
  pg1 = svptrue_b8();
  pg2 = svpfalse_b8();
  v_shift = svdup_n_s8(1);

  // Core SIMD Logic
  // ...
  svint8_t t1, t2;
  pg2 = svcmpeq(pg1, v_my, v_opp);
  // ...
  t1 = svlsl_x(pg2, v_my, v_shift);
  cand = svand_x(pg2, svand_x(pg2, svlsl_x(pg2, v_my,
              v_shift), v_opp), v_mask);
  // ...
  // ... more code ...
  // ...
}
```

SVE2の命令群は一見すると暗号の壁に見える。

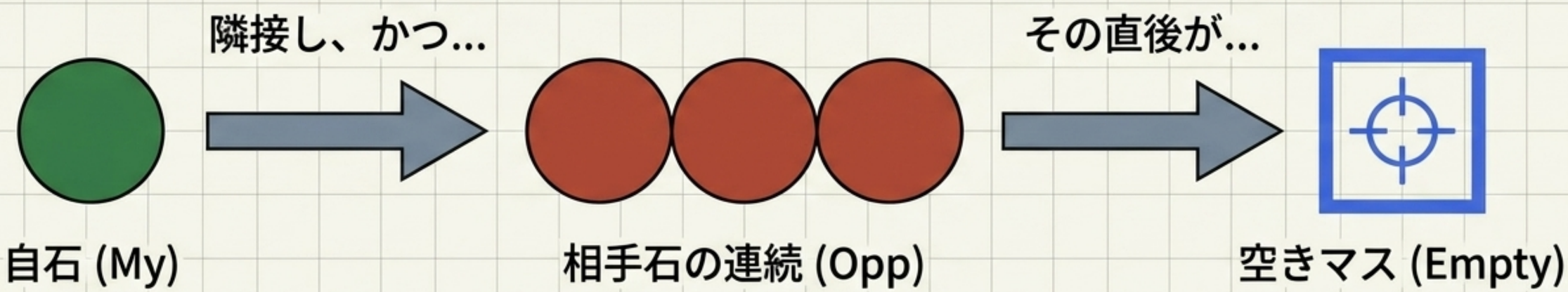


しかし、実際に行っているのは「自石の先にある相手石の連続をたどり、空きマスを見つける」というオセロの基本ルールに過ぎない。

本資料の目的：難解なSIMD命令を「盤面上の物理的な動き」へと完全に翻訳する。

NotebookLM

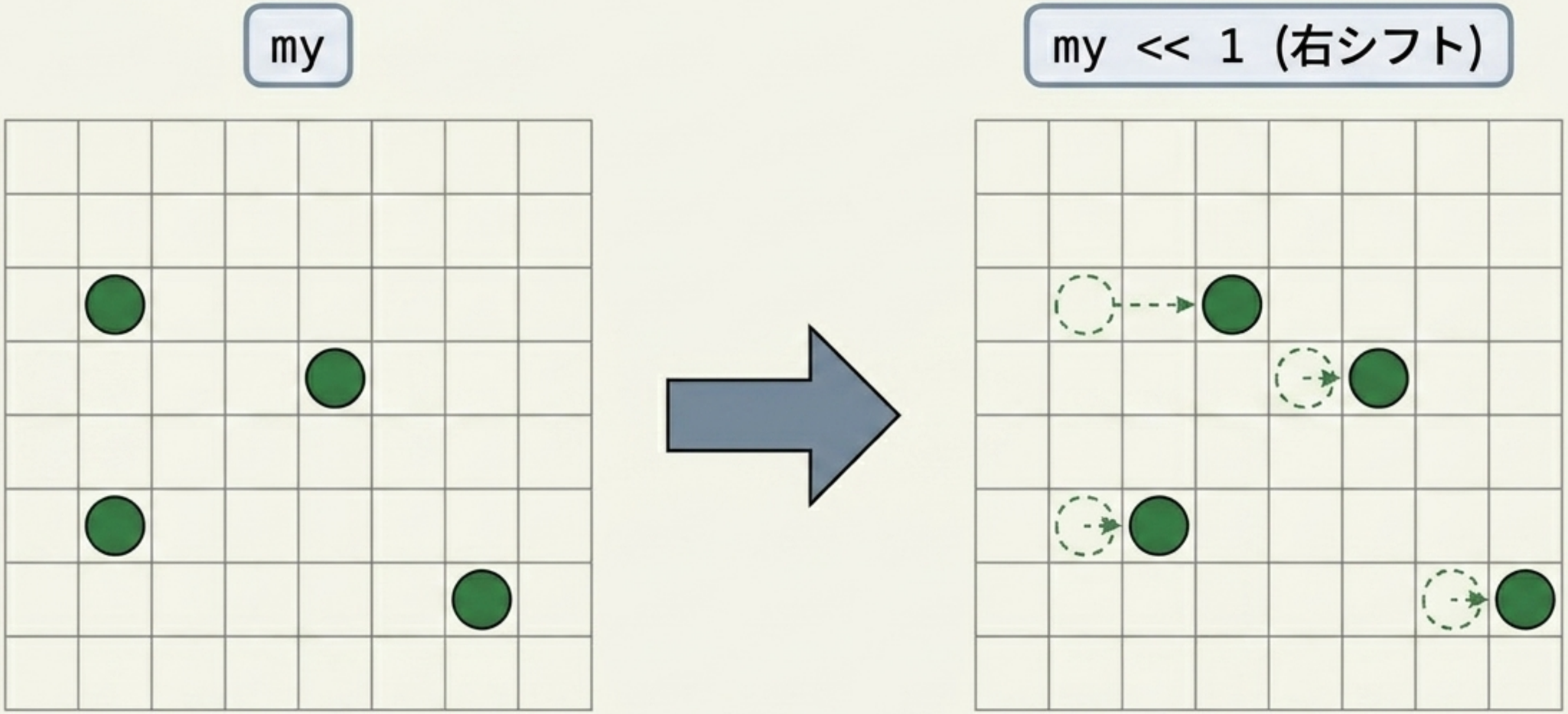
合法手の定義：3つの条件の連鎖



この「自石 → 相手石列 → 空きマス」という1方向のパターンを、
全マス・全方向同時に見つけるのがビット演算の役割である。

NotebookLM

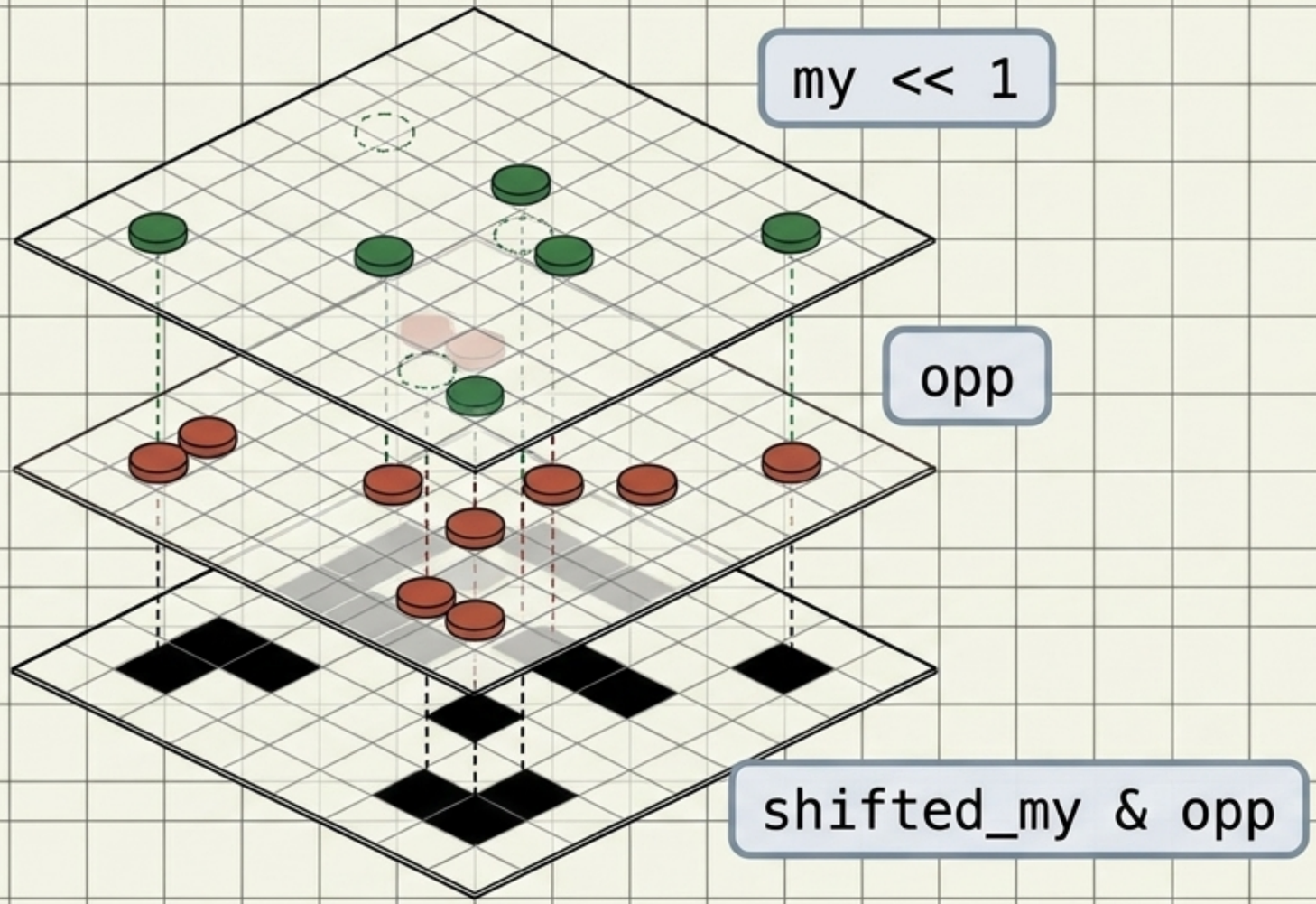
翻訳原理①：Shift = 盤面全体のスライド



ビットシフトは単なる数値計算ではない。64個の石すべてを特定の方向に「1マス同時に進める」物理的な操作である。（例：右方向は << 1、下方向は << 8）

NotebookLM

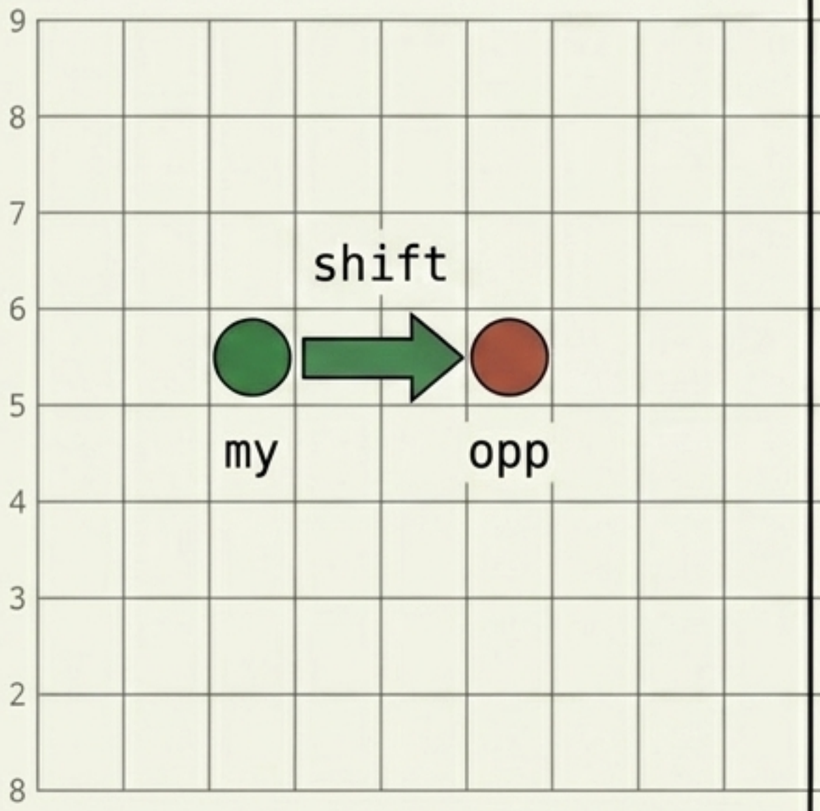
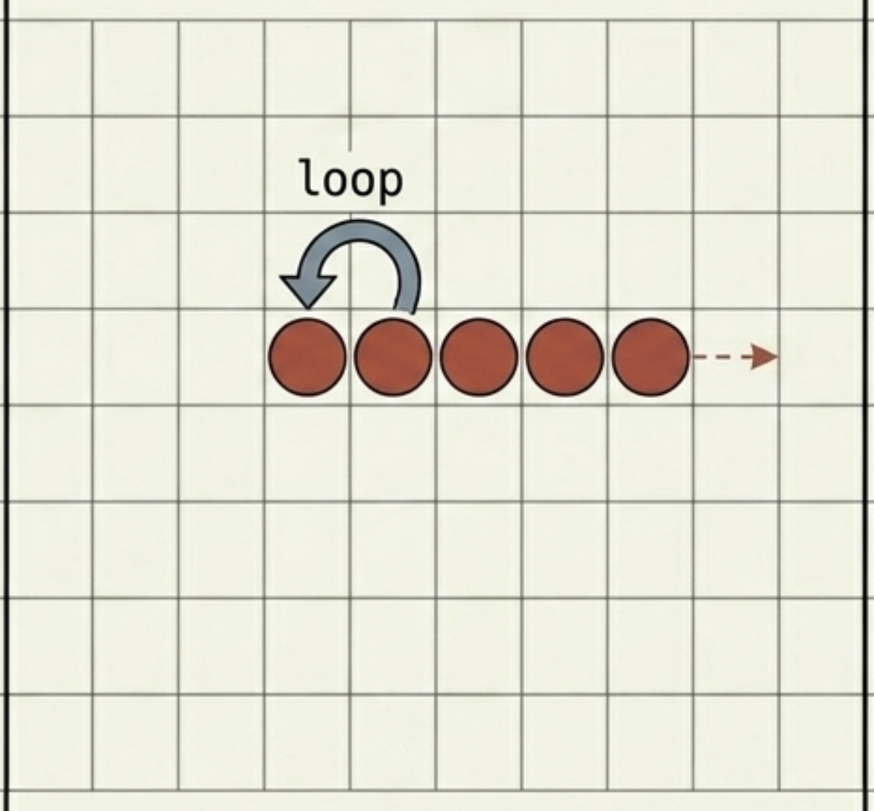
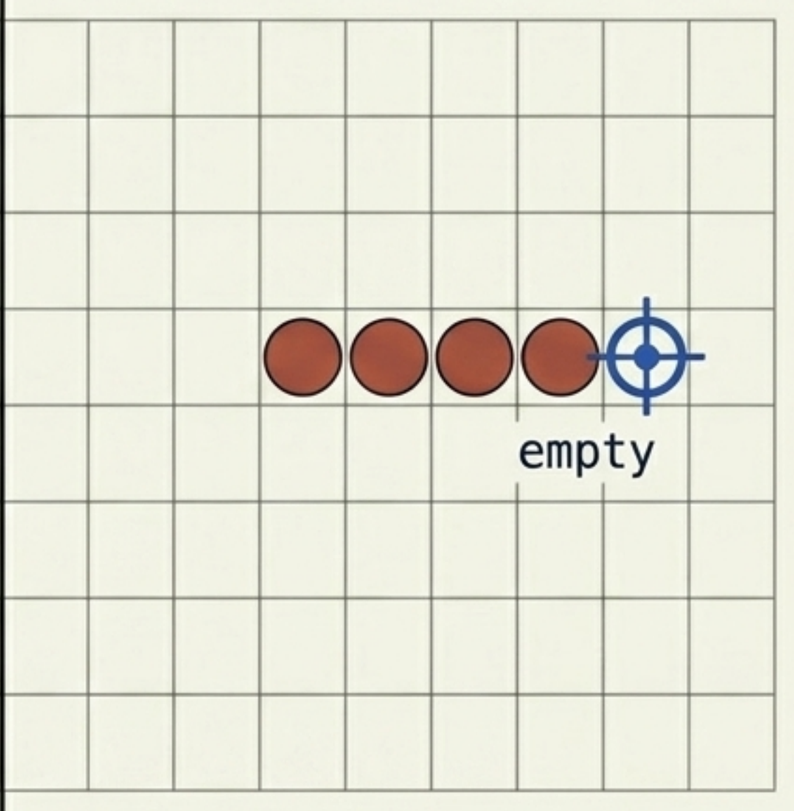
翻訳原理②：AND = 衝突判定



シフトした自石と相手石のビットANDをとる。結果が「1」に残ったマスは、「自石を1マス進めた先に相手石が存在する」ことを証明している。これが挟み判定の第一歩となる。

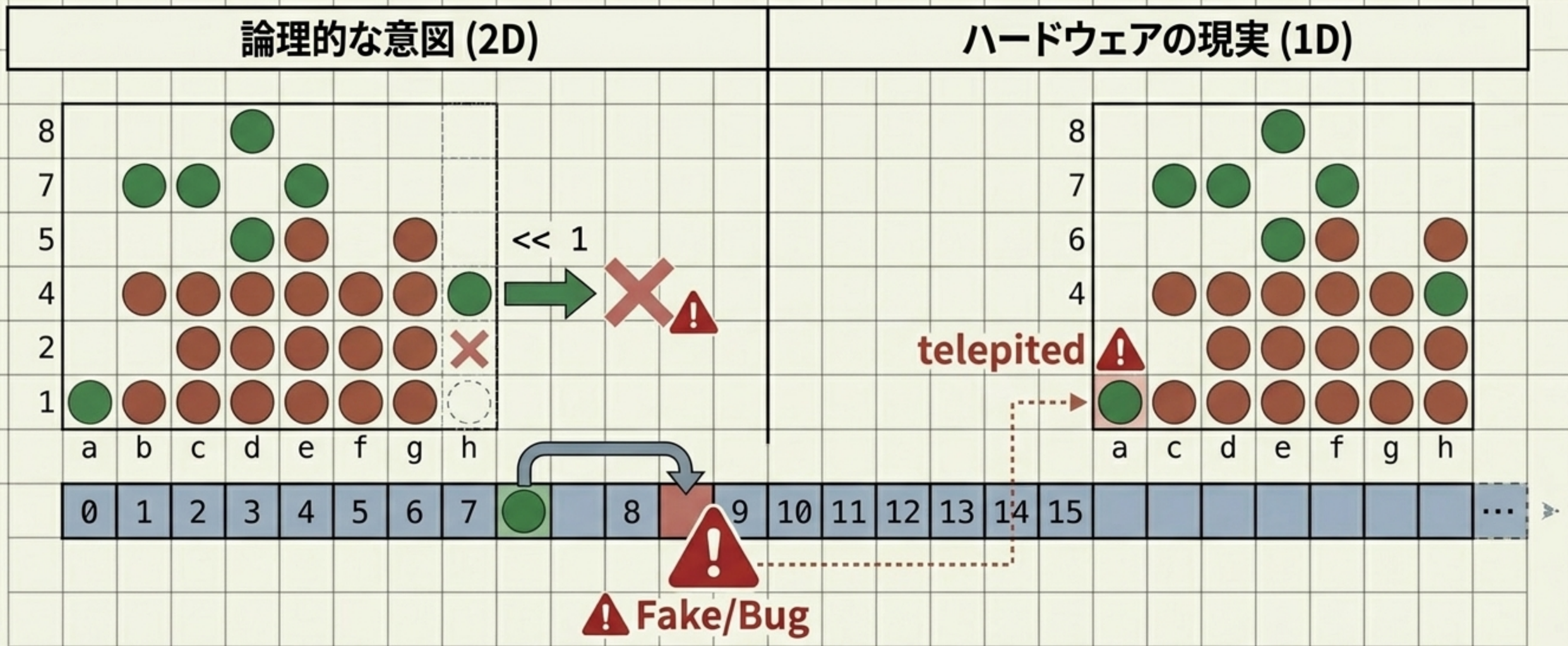
NotebookLM

合法手生成の3ステップ・エンジン

Step 1（隣接相手石の発見）	Step 2（相手石列の拡張）	Step 3（空きマスの確定）
		
自石を1マス進め、隣にある相手石を検出する。	発見した相手石をさらにシフトし続け、連続する相手石の「先」へとカーソルを進める (最大5回ループ)。	相手石列の先端を1マス進め、そこが「空きマス」であれば合法手として記録する。

NotebookLM

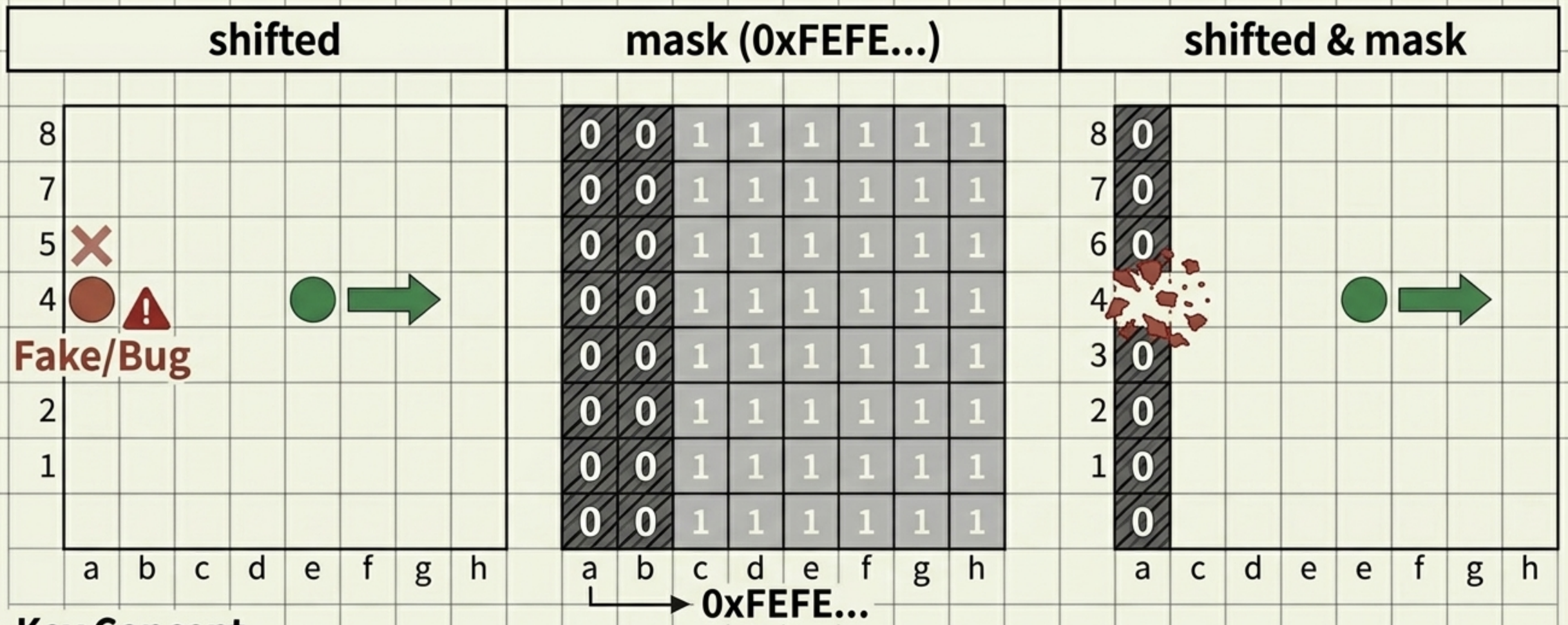
1D配列の罠：エッジ・ラップアラウンド



ビットボードは1次元配列であるため、盤面の右端(h列)から右シフトすると、物理的に「次の行の左端(a列)」へ回り込んでしまう。この偽ビットを放置すると誤った合法手が生成される。

NotebookLM

EDGE_MASK による「はみ出し」の物理的遮断



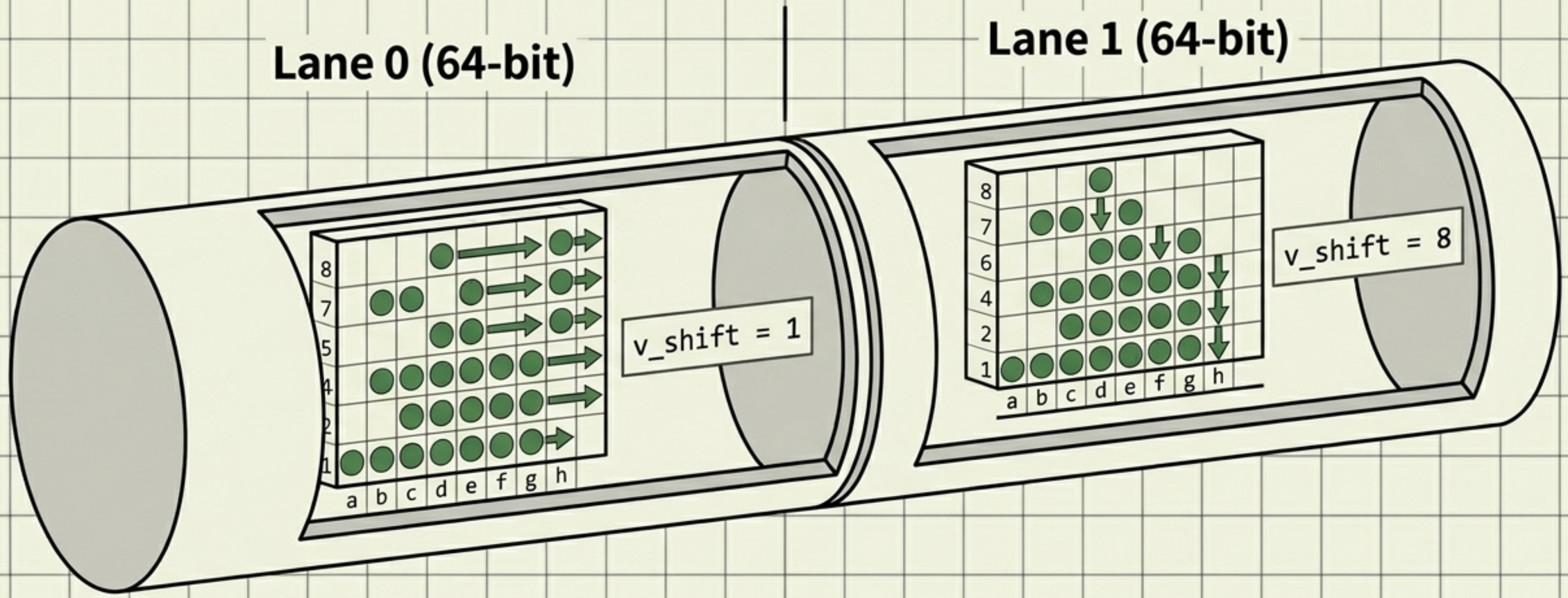
Key Concept

Post-Maskingの原則: マスクはシフト「前」ではなく、シフト「後」に適用する。

はみ出した先 (Destination) の列/行を 0 にすることで、回り込んだ偽ビットだけを正確に刈り取る。

NotebookLM

SVE2アーキテクチャ：64bit×2レーンの並列エンジン

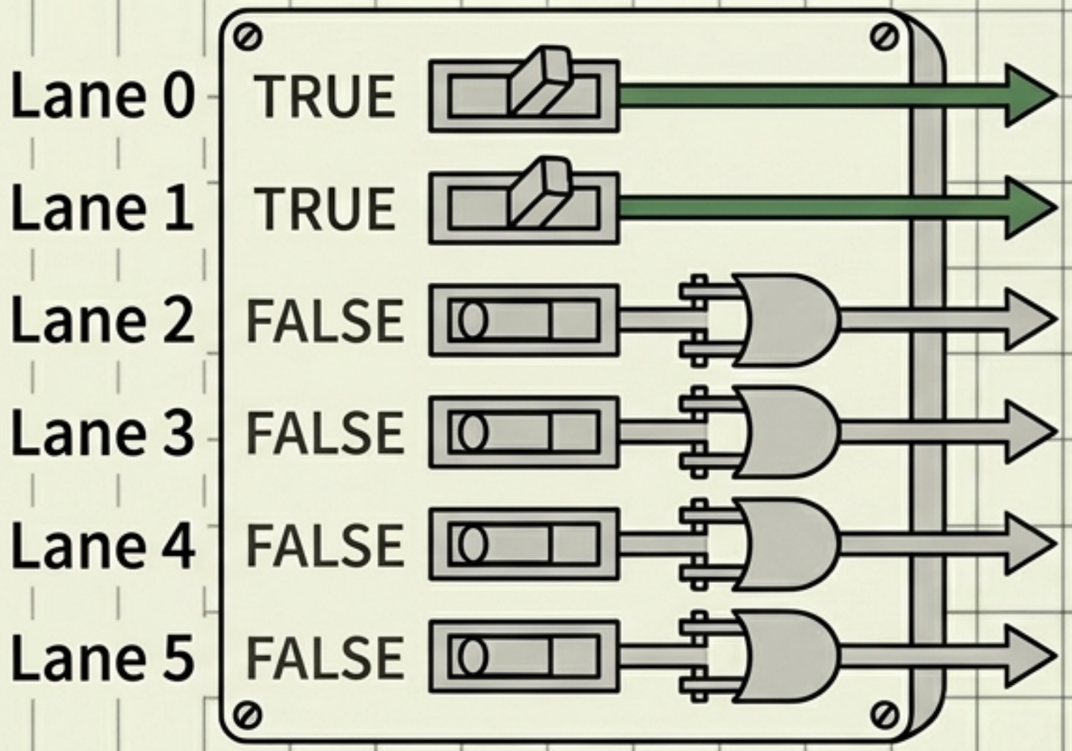


128ビットレジスタを「64ビット×2レーン」として扱う。1つのSVE2命令を発行するだけで、異なる2方向 (例：右と下) の盤面シフトを同時に、かつ完全に独立して実行する。8方向の探索されも、4ペアに分けることでループ回数が半減する。

NotebookLM

SIMD暗号の解読：述語(Predicate)とサフィックス

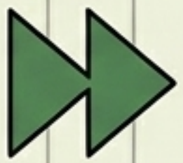
述語レジスタ pg (svbool_t)



命令サフィックスの法則



型指定 (_u64): 命令自体が「このレジスタを64ビット区切りで使え」とプロセッサに指示する（レーン幅の決定）。

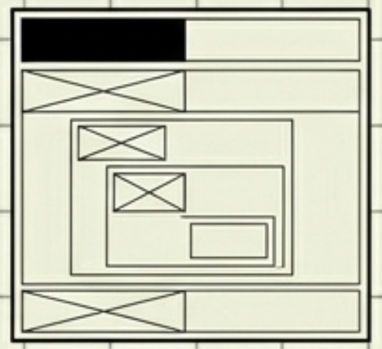


未定義許容(_x): svlsr_x など。述語がFALSEのレーンは「結果がゴミになっても構わない」と指定することで、最高速で演算を実行させるフラグ。

svwhilelt_b64(0, 2) は「先頭2レーンだけを処理しろ」という絶対命令（マスク）。
if文による条件分岐をハードウェアレベルでウェアレベルで排除する。
List 2では常に全レーン(2レーン)がTRUE。

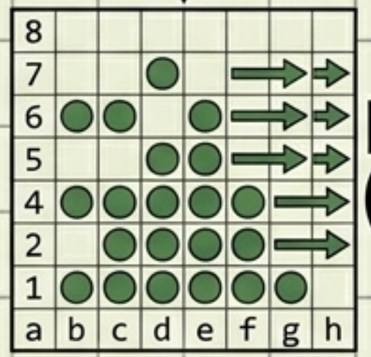
NotebookLM

Code X-Ray: Step 1 (起点と隣接相手石の検出)



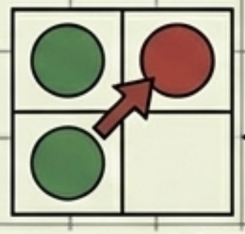
```
svuint64_t cand = svand_x(pg2,
                  svand_x(pg2,
                  svlsl_x(pg2, v_my, v_shift),
                  v_opp), v_mask);
```

1



自石の一斉シフト
(Movement)

2



相手石との衝突判定
(Collision)

3



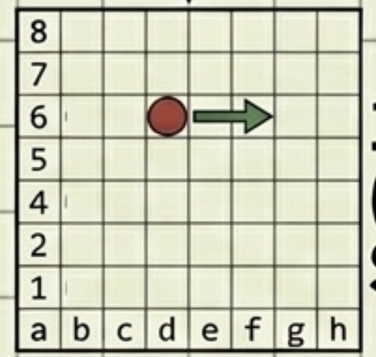
EDGE_MASKでの偽ビット消去
(Cleanup)

この1行のネストされた命令で、2方向同時に「自石のすぐ隣にある相手石」だけを抽出したビットボード (cand) が完成。学 べで、自石の すぐ隣にある相手石」だけを抽出したビットボード(cand)が完成する。

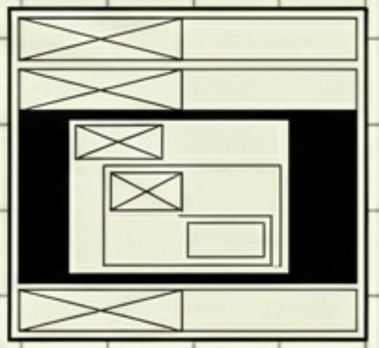
NotebookLM

Code X-Ray: Step 2 (相手石列の拡張ループ)

```
for (int rep = 0; rep < 5; rep++) {
  svuint64_t next = svand_x(pg2,
                    svand_x(pg2,
                    svlsl_x(pg2, cand, v_shift),
                    v_opp), v_mask);
  cand = svorr_x(pg2, cand, next);
}
```

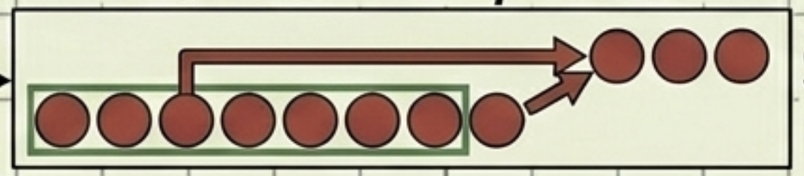


1マス先へ追跡
(Track One Square Ahead)



Minimap

Accumulator/Stack

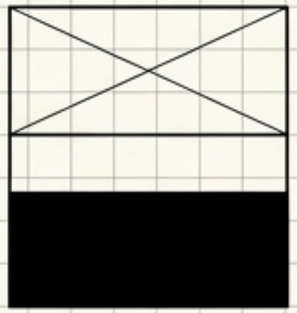


道がすべて蓄積される
(All Paths Are Accumulated)

1ループごとに相手石列を1マス先へ追跡する。最大6マスの連続まで対応。
cand には発見された「相手石の道」がすべて蓄積される。

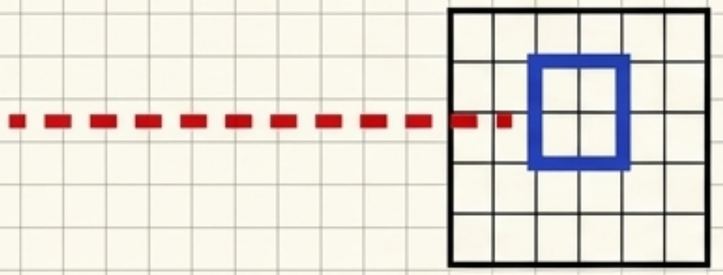
NotebookLM

Code X-Ray: Step 3 & リダクション (合法手確定)

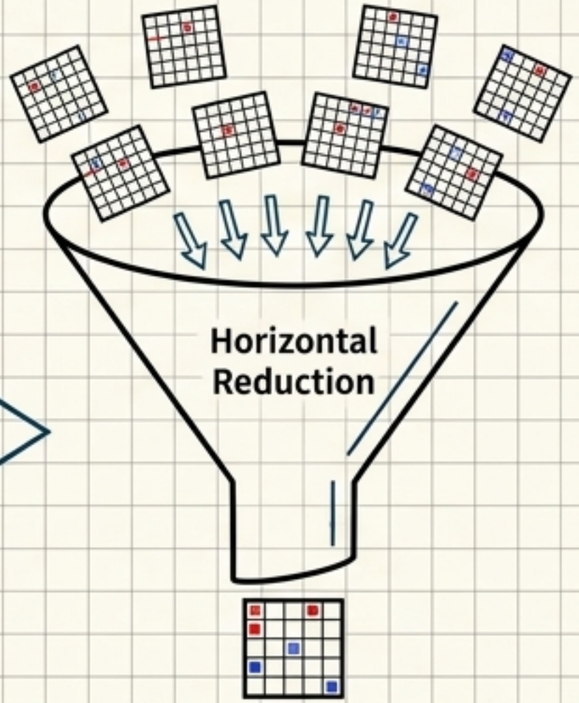



```
svuint64_t hit = svand_x(pg2,
                         svand_x(pg2,
                           svlsl_x(pg2, cand, v_shift),
                                v_empty), v_mask);

return svorv_u64(pg2, v_legal);
```



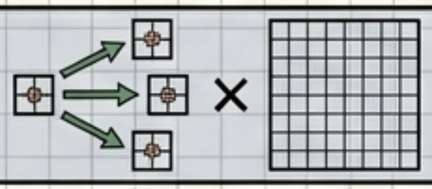
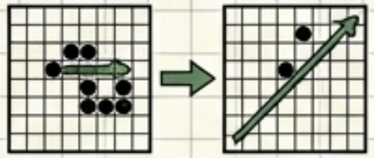
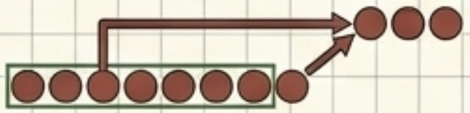
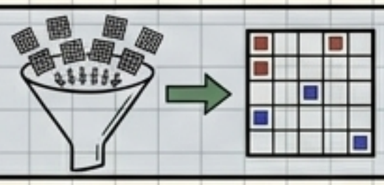
合法手の誕生



空きマスとのANDで合法手が確定。最後に svorv_u64 が、ベクトルレジスタ内に散らばった全８方向の結果を「１命令」で１つの64ビット整数に統合する。

NotebookLM

総合リファレンス：SVE2 オセロ語彙表

SVE2命令	盤面上の幾何学的意味	コードでの役割
svdup_u64		盤面全体を各レーンへブロードキャスト（配置）
svlsl_x / svlsr_x		全石を一斉に指定方向(1, 7, 8, 9)へ移動させる
svand_x		重なり合う石（相手石や空きマス）の検出、 または盤外はみ出しの消去
svorr_x		探索した相手石の「道」を記録・結合する
svorv_u64		全方向の探索結果ボードを、 １枚の最終合法手ボードに統合する

List 2 は魔術ではない。64マス同時に動く、並列化された物理エンジンの設計図である。

NotebookLM